

Technical Assessment: Software Testing Skills

Review the code and scenarios below. Identify bugs, write test cases, and answer the questions.

There are no hints — read carefully.

Deadline: 5 days from receipt

Format: Single PDF or Google Doc

Total marks: 100

Task 1 — Node.js Bug Hunt **Node.js** 25 marks

Find all bugs in this user registration function **tricky**

This function registers a new user. It was deployed to staging and is causing issues. Find every bug — logical, security, and runtime.

```
// POST /api/register
async function registerUser(req, res) {
  const { username, email, password } = req.body;

  if (!username || !email || !password) {
    res.status(400).json({ error: "All fields required" });
  }

  const existing = await db.query(
    `SELECT * FROM users WHERE email = '${email}'`
  );

  if (existing.rows.length > 0) {
    return res.status(409).json({ error: "Email already exists" });
  }

  const hashedPassword = await bcrypt.hash(password, 1);

  const newUser = await db.query(
    `INSERT INTO users (username, email, password)
    VALUES ($1, $2, $3) RETURNING id`,
    [username, email, hashedPassword]
  );

  return res.status(200).json({ userId: newUser.rows[0].id });
}
```

List every bug you find. For each one, state: what the bug is, what can go wrong because of it, and how you would fix it.

How many bugs are there in total? (Hint: count carefully — there are more than 2)

Deliverable

Numbered list of bugs with explanation + corrected code snippet

Task 2 — React Component Testing **React 25 marks**

What is wrong with this component? **tricky**

A developer built a search input that filters a user list. Users report: it searches correctly once, but then stops working. Sometimes it crashes. Find all issues.

```
function UserSearch({ users }) {  
  const [query, setQuery] = React.useState("");  
  const [results, setResults] = React.useState(users);  
  
  const handleSearch = () => {  
    const filtered = users.filter(u => {  
      return u.name.includes(query);  
    });  
    setResults(filtered);  
  };  
  
  React.useEffect(() => {  
    handleSearch();  
  }, []);  
  
  return (  
    <div>  
      <input  
        value={query}  
        onChange={(e) => setQuery(e.target.value)}  
        placeholder="Search users..."  
      />  
      <button onClick={handleSearch}>Search</button>  
      {results.map(user => (  
        <div key={user.id}>{user.name}</div>  
      ))}  
    </div>  
  );  
}
```

Why does search stop working after the first search in some cases?

What crashes can happen and under what data condition?

The search is case-sensitive — is that a bug or a feature? How would you test for it?

Write 3 test cases for this component (name, steps, expected result format)

Deliverable

Bug list + 3 structured test cases (TC ID, description, steps, expected result, pass/fail criteria)

Task 3 — REST API Testing **REST API 25 marks**

Design test cases for this API endpoint

You are given the following API contract. No code is shown — test the contract itself.

Endpoint: `PATCH /api/users/:id`
Purpose: Update a user's profile (name, email, or both)
Auth: Bearer token required
Rules:

- Email must be unique across all users
- Email must be a valid format
- Name must be 2-50 characters
- Cannot update another user's profile (only own)
- Returns 200 on success with updated user object
- Returns appropriate error codes for all failure cases

Write a minimum of 8 test cases covering: happy path, edge cases, security, and error handling

What HTTP status codes do you expect — list them all with the scenario each one maps to

One test case must check what happens when the token belongs to User A but the `:id` in the URL is User B's ID. What should happen?

What would you send as the request body to attempt an SQL injection? (Write it out — this tests your understanding of attack surfaces)

Deliverable

Table with columns: TC ID | Scenario | Input | Expected Status Code | Expected Response | Pass/Fail

Task 4 — PostgreSQL Query Analysis PostgreSQL 25 marks

What does this query actually return? **very tricky**

Given the tables and data below, predict exactly what each query returns. Do not run it — reason through it.

```
-- Table: orders
-- id | user_id | amount | status
-- 1  | 1      | 500    | 'completed'
-- 2  | 1      | 200    | 'pending'
-- 3  | 2      | 800    | 'completed'
-- 4  | 3      | 150    | NULL
```

```
-- Table: users
-- id | name   | active
-- 1  | Ravi   | true
-- 2  | Meena  | true
-- 3  | Suresh | false
```

```
-- Query A
SELECT u.name, SUM(o.amount) AS total
```

```
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.status = 'completed'
GROUP BY u.name;

-- Query B
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.name
HAVING COUNT(o.id) > 1;

-- Query C (bug planted here)
SELECT * FROM orders
WHERE status != 'pending';
```

What rows does Query A return? Does it return all users? Why or why not?

What rows does Query B return? What does the HAVING clause do here?

Query C is intended to return all non-pending orders. Does it? What is the bug and what fix would you apply?

How would you write a query to find users who have never placed an order?

Deliverable

For each query: predicted result table + explanation of your reasoning